

```

import type { RealtimeChannel, Session } from "@supabase/supabase-js";
import { useCallback, useEffect, useRef, useState } from "react";

import { supabase } from "@integrations/supabase/client";

type Signal = {
  from: string;
  to: string;
  description?: RTCSessionDescriptionInit;
  candidate?: RTCIceCandidateInit;
};

const rtcConfig: RTCConfiguration = {
  iceServers: [{ urls: "stun:stun.l.google.com:19302" }],
};

export function useVoiceRoom(roomId: string, session: Session | null, enabled: boolean) {
  const [muted, setMuted] = useState(true);
  const [error, setError] = useState("");
  const streamRef = useRef<MediaStream | null>(null);
  const channelRef = useRef<RealtimeChannel | null>(null);
  const peersRef = useRef(new Map<string, RTCPeerConnection>());
  const audioRef = useRef(new Map<string, HTMLAudioElement>());

  const closePeer = useCallback((userId: string) => {
    peersRef.current.get(userId)?.close();
    peersRef.current.delete(userId);
    const audio = audioRef.current.get(userId);
    if (audio) {
      audio.srcObject = null;
      audio.remove();
    }
    audioRef.current.delete(userId);
  }, []);

  useEffect(() => {
    if (!enabled || !session) return;
    let active = true;
    const myId = session.user.id;

    const sendSignal = async (payload: Omit<Signal, "from">) => {
      await channelRef.current?.send({
        type: "broadcast",

```

```

    event: "webrtc-signal",
    payload: { ...payload, from: myId },
  });
};

const createPeer = (remoteId: string) => {
  const existing = peersRef.current.get(remoteId);
  if (existing) return existing;
  const peer = new RTCPeerConnection(rtcConfig);
  streamRef.current?.getTracks().forEach((track) => {
    const stream = streamRef.current;
    if (stream) peer.addTrack(track, stream);
  });
  peer.onicecandidate = ({ candidate }) => {
    if (candidate) void sendSignal({ to: remoteId, candidate: candidate.toJSO
  });
  peer.ontrack = ({ streams }) => {
    const remoteStream = streams[0];
    if (!remoteStream) return;
    let audio = audioRef.current.get(remoteId);
    if (!audio) {
      audio = document.createElement("audio");
      audio.autoplay = true;
      audio.setAttribute("playsinline", "true");
      audio.setAttribute("aria-hidden", "true");
      document.body.appendChild(audio);
      audioRef.current.set(remoteId, audio);
    }
    audio.srcObject = remoteStream;
    void audio.play().catch(() => setError("Sesi duymak için ekrana bir kez d
  });
  peer.onconnectionstatechange = () => {
    if (["failed", "closed"].includes(peer.connectionState)) closePeer(remote
  );
  peersRef.current.set(remoteId, peer);
  return peer;
};

const makeOffer = async (remoteId: string) => {
  const peer = createPeer(remoteId);
  const offer = await peer.createOffer();
  await peer.setLocalDescription(offer);
  await sendSignal({ to: remoteId, description: offer });
};

const channel = supabase.channel(`voice-${roomId}`, {

```

```

    config: { presence: { key: myId } },
  });
  channelRef.current = channel;
  channel
    .on("broadcast", { event: "webrtc-signal" }, async ({ payload }) => {
      const signal = payload as Signal;
      if (!active || signal.to !== myId || signal.from === myId) return;
      try {
        const peer = createPeer(signal.from);
        if (signal.description) {
          await peer.setRemoteDescription(signal.description);
          if (signal.description.type === "offer") {
            const answer = await peer.createAnswer();
            await peer.setLocalDescription(answer);
            await sendSignal({ to: signal.from, description: answer });
          }
          else if (signal.candidate) {
            await peer.addIceCandidate(signal.candidate);
          }
        }
        catch {
          setError("Ses bağlantısı kurulamadı. Odaya yeniden girmeyi dene.");
        }
      })
    }
    .on("presence", { event: "join" }, ({ key }) => {
      if (key !== myId && myId.localeCompare(key) < 0) void makeOffer(key);
    })
    .on("presence", { event: "leave" }, ({ key }) => closePeer(key))
    .subscribe(async (status) => {
      if (status !== "SUBSCRIBED") return;
      await channel.track({ user_id: myId });
    });

  return () => {
    active = false;
    channelRef.current = null;
    void channel.untrack();
    void supabase.removeChannel(channel);
    [...peersRef.current.keys()].forEach(closePeer);
    streamRef.current?.getTracks().forEach((track) => track.stop());
    streamRef.current = null;
    setMuted(true);
  };
}, [closePeer, enabled, roomId, session]);

const toggleMic = useCallback(async () => {
  setError("");

```

```

try {
  let stream = streamRef.current;
  if (!stream) {
    stream = await navigator.mediaDevices.getUserMedia({
      audio: { echoCancellation: true, noiseSuppression: true, autoGainContro
    });
    streamRef.current = stream;
    const activeStream = stream;
    peersRef.current.forEach((peer, remoteId) => {
      activeStream.getTracks().forEach((track) => peer.addTrack(track, active
    void (async () => {
      const offer = await peer.createOffer();
      await peer.setLocalDescription(offer);
      await channelRef.current?.send({
        type: "broadcast",
        event: "webrtc-signal",
        payload: { from: session?.user.id ?? "", to: remoteId, description:
      });
    })();
  });
}
const nextMuted = !muted;
stream.getAudioTracks().forEach((track) => { track.enabled = !nextMuted; })
setMuted(nextMuted);
} catch {
  setError("Mikrofon izni verilmedi. Tarayıcı ayarlarından izin verip tekrar
}
}, [muted]);

return { muted, error, toggleMic };
}

```